

Distributed Systems

Lecture18: Mutual Exclusion

Mohammad Reza Shakournia

shakournia@ut.ac.ir

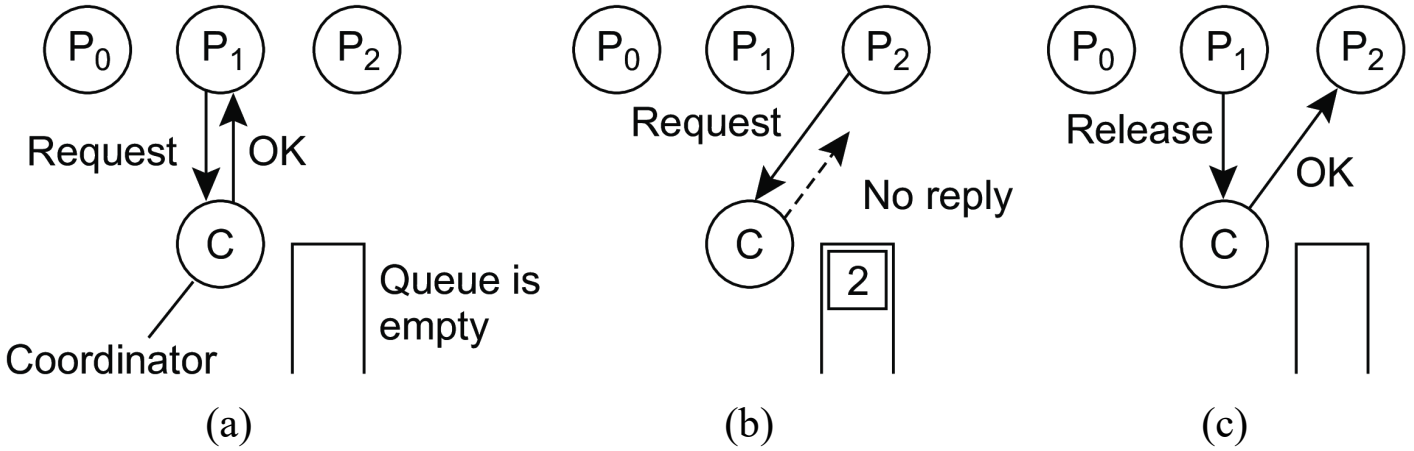
School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Mutual Exclusion

- Several processes in a distributed system want exclusive access to some resource.
- **Basic solutions:**
  - Permission-based: A process wanting to enter its critical region, or access a resource, needs permission from other processes.
  - Token-based: A token is passed between processes. The one who has the token may proceed in its critical region, or pass it on when not interested.

Permission-based, Centralized

- Simply use a coordinator



a) Process P_1 asks the coordinator for permission to access a shared resource. Permission is granted.

b) Process P_2 then asks permission to access the same resource. The coordinator does not reply.

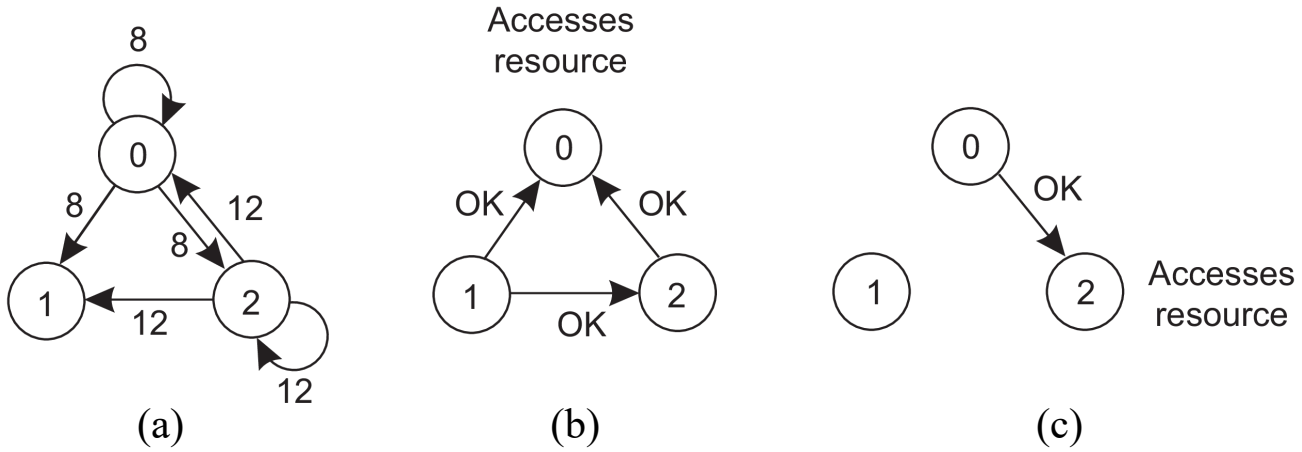
c) When P_1 releases the resource, it tells the coordinator, which then replies to P_2.

Mutual Exclusion: Ricart & Agrawala

- The same as Lamport except that acknowledgments are not sent
- Return a response to a request only when:
  - The receiving process has no interest in the shared resource
  - The receiving process is waiting for the resource, but has lower priority (known through comparison of timestamps).
- In all other cases, reply is deferred, implying some more local administration.

Mutual Exclusion: Ricart & Agrawala

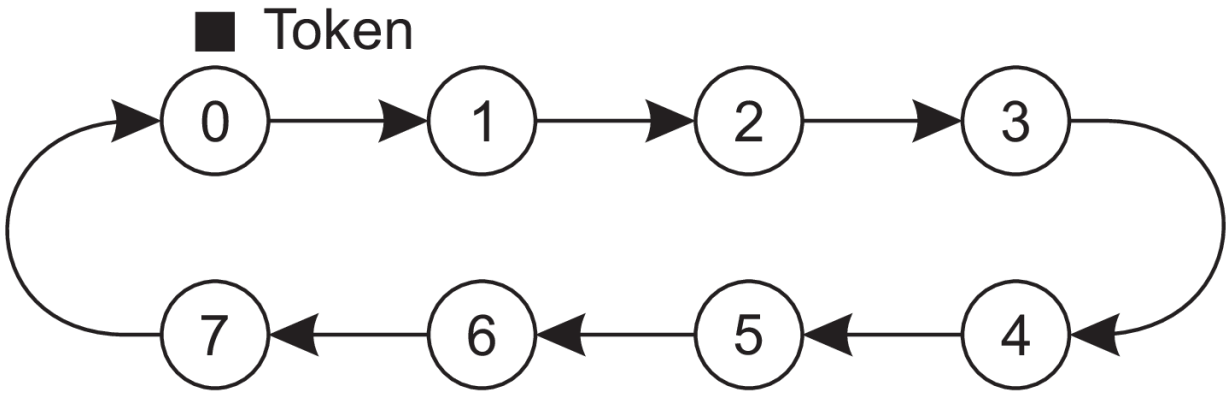
- **Example with three processes**



a) Two processes want to access a shared resource at the same moment.
b) P_0 has the lowest timestamp, so it wins.
c) When process P_0 is done, it sends an *OK* also, so P_2 can now go ahead.

Mutual Exclusion: Token Ring Algorithm

- Organize processes in a logical ring, and let a token be passed between them. The one that holds the token is allowed to enter the critical region (if it wants to).
- An overlay network constructed as a logical ring with a circulating token



Decentralized Mutual Exclusion

- **Principle**
  - Assume every resource is replicated N times, with each replica having its own coordinator ⇒ access requires a majority vote from m > N/2 coordinators. A coordinator always responds immediately to a request.
- **Assumption**
  - When a coordinator crashes, it will recover quickly, but will have forgotten about permissions it had granted.

Decentralized Mutual Exclusion

- How robust is this system?
  - Let $p = \Delta t/T$ be the probability that a coordinator resets during a time interval Δt, while having a lifetime of T.
  - The probability P[k] that k out of m coordinators reset during the same interval is

$$\mathbb{P}[k] = \binom{m}{k} p^k (1-p)^{m-k}$$

  - f coordinators reset $\Rightarrow$ correctness is violated when there is only a minority of nonfaulty coordinators: when $N - (m - f) \geq m$, or, $f \geq 2m - N$.
  - The probability of a violation is $\sum_{k=2m-N}^{m} \mathbb{P}[k]$

Decentralized Mutual Exclusion

- Violation probabilities for various parameter values

N	m	p	Violation
8	5	3 sec/hour	$< 10^{-5}$
8	6	3 sec/hour	$< 10^{-11}$
16	9	3 sec/hour	$< 10^{-4}$
16	12	3 sec/hour	$< 10^{-21}$
32	17	3 sec/hour	$< 10^{-4}$
32	24	3 sec/hour	$< 10^{-43}$

N	m	p	Violation
8	5	30 sec/hour	$< 10^{-3}$
8	6	30 sec/hour	$< 10^{-7}$
16	9	30 sec/hour	$< 10^{-2}$
16	12	30 sec/hour	$< 10^{-13}$
32	17	30 sec/hour	$< 10^{-2}$
32	24	30 sec/hour	$< 10^{-27}$

- So.... What can we conclude?

Mutual Exclusion: Comparison

Algorithm	Messages per entry/exit	Delay before entry (in message times)
Centralized	3	2
Distributed	$2 \cdot (N-1)$	$2 \cdot (N-1)$
Token ring	$1, \ldots, \infty$	$0, \ldots, N-1$
Decentralized	$2 \cdot m \cdot k + m, k = 1, 2, \ldots$	$2 \cdot m \cdot k$

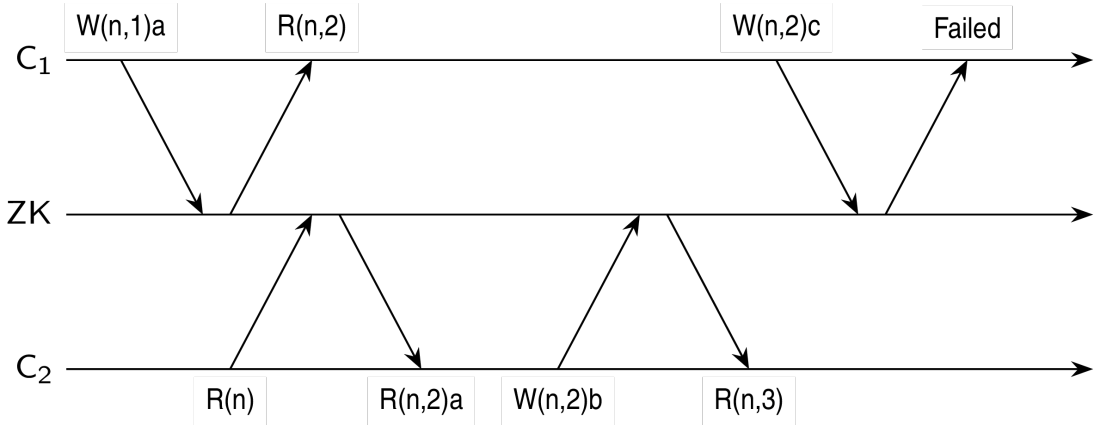
Example: ZooKeeper

- Basics (and keeping it simple)
  - Centralized server setup
  - All client-server communication is nonblocking: a client immediately gets a response
  - ZooKeeper maintains a tree-based namespace, akin to that of a filesystem
  - Clients can create, delete, or update nodes, as well as check existence.

Zookeeper Race Condition

- ZooKeeper allows a client to be notified when a node, or a branch in the tree, changes. This may easily lead to race conditions.
- Consider a simple locking mechanism
  1. A client C1 creates a node /lock .
  2. A client C2 wants to acquire the lock but is notified that the associated node already exists.
  3. Before C2 subscribes to a notification, C1 releases the lock, i.e., deletes /lock .
  4. Client C2 subscribes to changes to /lock and blocks locally.
- Solution: Use version numbers and check node’s status

Zookeeper Versioning



- **Notations**
  - W (n, k)a: request to write a to node n, assuming current version is k.
  - R(n, k): current version of node n is k .
  - R(n): client wants to know the current value of node n
  - R(n, k)a: value a from node n is returned with its current version k .

Zookeeper Locking Protocol

- It is now very simple
  1. lock: A client C1 creates a node /lock .
  2. lock: A client C2 wants to acquire the lock but is notified that the associated node already exists ⇒ C2 subscribes to notification on changes of /lock .
  3. unlock: Client C1 deletes node /lock ⇒ all subscribers to changes are notified.

Wrap-Up + What's Next

In the next lecture, we will discuss the Election Algorithms, including:

- Bully Algorithm,
- Ring Algorithm,
- Election in Large Scale System,
- Election in Wireless Environments.